MOBILE APPLICATION DEVELOPMENT

ASSIGNMENT #2: FULL-STACK API INTEGRATION

Project Name: Leo's Hub – Food Delivery System

Submitted By: Yashfa Javed

Registration #9066

Instructor: Ma,am Sadaf

Date: March 16, 2026

ABASYN UNIVERSITY

ISLAMABAD

BSCS:Year(Spring_2024)

Project Architecture (Intro)

1. Overview:

The objective of this assignment was to demonstrate the integration of a React Native mobile application with multiple Node.js backend services. The system is designed using a Micro services Architecture, where different backend servers handle specific tasks like Menu Management and Order Logging.

2. System Components:

- **Frontend:** Developed in React Native using Expo.
- **Backend A (Menu Server):** A Node.js server running on Port 5050 to provide real-time menu data in JSON format.
- **Backend B (Order Logger):** A Node.js server running on Port 8080 to receive and log customer orders via POST requests.

2.1.Technical Specifications:

- **Language:** JavaScript (ES6+)
- **Framework:** React Native with Expo CLI
- **Backend Runtime:** Node.js
- **Communication Protocol:** HTTP/HTTPS using REST Principles
- **Data Format:** JSON (JavaScript Object Notation)

2.2 Development Workflow & Network Logic:

The application operates on a local area network (LAN) environment. To ensure seamless data flow between the laptop (acting as the server) and the Android/iOS device (acting as the client), static IP bridging was implemented. This eliminates the "Connection Refused" errors typically found in localhost-only environments.

2.3 Key Features Implemented:

- **Dynamic Menu Fetching:** Instead of hardcoding data, the app requests the latest menu from the Port 5050 server during the useEffect lifecycle hook.
- **Asynchronous Order Processing:** Orders are sent as asynchronous POST requests, ensuring the UI remains responsive while the server logs the data.

- **Multi-Port Listening:** Simultaneous handling of two different server instances to separate data retrieval from transaction logging.

3.System Logic Flow:

Component	Port	Protocol	Action	DataFlow
Mobile App (Client)	N/A	HTTP/GET	Request Menu	GET/Menu
Menu Server(Server1)	5050	HTTP	Serve Data	JSON Menu List
Mobile App (Client)	N/A	HTTP/POST	Place Order	Order Payload(JSON)
Order Logger(Server2)	8080	HTTP	Log Transaction	Status 200 OK
Terminal(Monitor)	N/A	Console	Output Logs	“Ner Order Received!”

❖ Implementation Details:

3.1 BackendService A: Menu API (Port 5050)

Dynamic Menu Provider:

The Menu Service was developed using Node.js and the Express.js framework. Its primary purpose is to decouple the data layer from the frontend, providing a dynamic source for food items rather than using static hardcoded arrays.

Port Configuration: The server is configured to listen on Port 5050.

Endpoint Logic: It exposes a GET /menu endpoint. Upon receiving a request, the server responds with a JSON array containing object properties such as Id, name, price, and ImageUrl.

Asset Management: Food images are served via external high-resolution URLs to optimize the application's local storage performance.

Pretty-print


```
  {
    "id": 4,
    "name": "Grilled Club Sandwich",
    "price": 350,
    "category": "Daily Specials",
    "imageUrl": "https://images.unsplash.com/photo-1528735602780-2552fd46c7af?w=500"
  },
  {
    "id": 5,
    "name": "Loaded Cheese Fries",
    "price": 300,
    "category": "Sides",
    "imageUrl": "https://images.unsplash.com/photo-1573080496219-bb080dd4f877?w=500"
  },
  {
    "id": 6,
    "name": "Chocolate Fudge Shake",
    "price": 400,
    "category": "Beverages",
    "imageUrl": "https://images.unsplash.com/photo-1572490122747-3968b75cc699?w=500"
  },
  {
    "id": 7,
    "name": "Garden Fresh Salad",
    "price": 250,
    "category": "Healthy",
    "imageUrl": "https://images.unsplash.com/photo-1512621776951-a57141f2eefd?w=500"
  },
  {
    "id": 8,
    "name": "Spicy Pasta",
    "price": 800,
    "category": "Italian",
    "imageUrl": "https://images.unsplash.com/photo-1563379091339-03b21bc4a4f8?w=500"
  }
]
```


Pretty-print


```
[
  {
    "id": 1,
    "name": "Premium Pepperoni Pizza",
    "price": 1200,
    "category": "Daily Specials",
    "imageUrl": "https://images.unsplash.com/photo-1513104890138-7c749659a591?w=500"
  },
  {
    "id": 2,
    "name": "Classic Zinger Burger",
    "price": 450,
    "category": "Fast Food",
    "imageUrl": "https://images.unsplash.com/photo-1568901346375-23c9450c58cd?w=500"
  },
  {
    "id": 3,
    "name": "Authentic Kashmiri Chai",
    "price": 150,
    "category": "Beverages",
    "imageUrl": "https://images.unsplash.com/photo-1571935441005-46704044814e?w=500"
  },
  {
    "id": 4,
    "name": "Grilled Club Sandwich",
    "price": 350,
    "category": "Daily Specials",
    "imageUrl": "https://images.unsplash.com/photo-1528735602780-2552fd46c7af?w=500"
  },
  {
    "id": 5,
    "name": "Loaded Cheese Fries",
    "price": 300,
    "category": "Sides",
    "imageUrl": "https://images.unsplash.com/photo-1573080496219-bb080dd4f877?w=500"
```

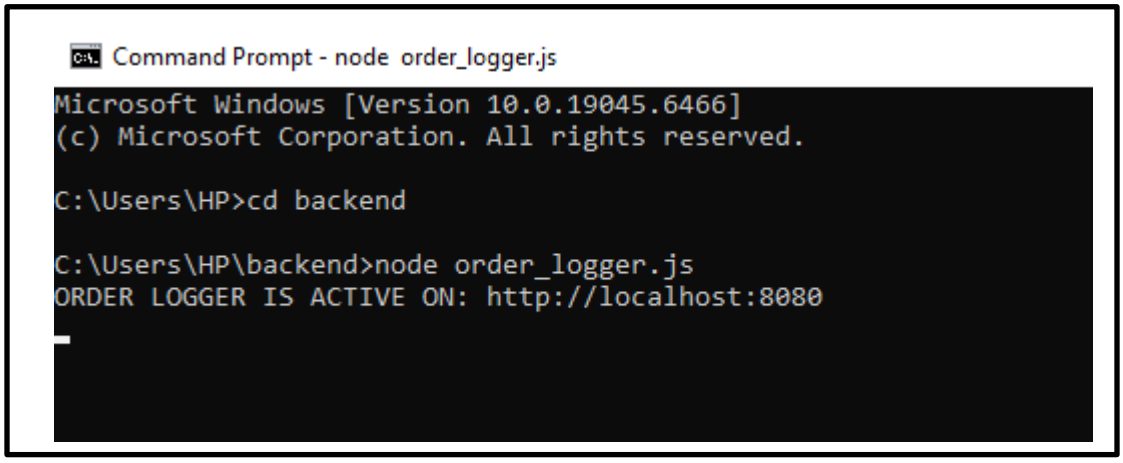
3.2 Backend Service B: Transactional Order Logger

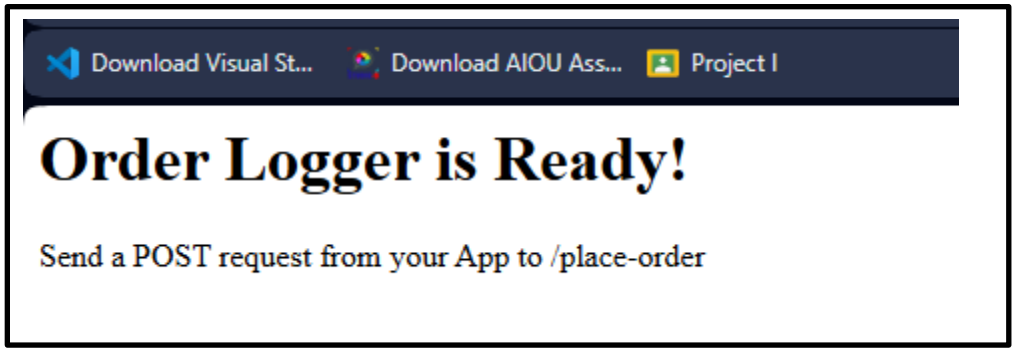
The Order Logger is a dedicated micro service designed to capture and process customer transactions in real-time.

Port Configuration: This service operates independently on Port 8080.

Endpoint Logic: It utilizes a POST /place-order endpoint. When a user triggers the "Place Order" action, the app transmits a payload containing UserName, Items, Amount, and Payment Method.

Server-Side Logging: The server parses the incoming JSON request and outputs the transaction details directly to the console/terminal for verification.





3.3 Frontend-Backend Connectivity (The Integration Bridge):

The React Native frontend utilizes the native fetch() API for network communication. To ensure successful integration, the following networking protocols were implemented:

Network Bridging: Both the physical mobile device and the host laptop were connected to the same Local Area Network (LAN).

Static IP Addressing: To bypass the limitations of localhost on mobile devices, the server's internal IP address (192.168.0.113) was utilized in the API calls.

Asynchronous Logic: The implementation uses async/await syntax to handle network latency, ensuring a smooth and non-blocking User Experience (UX).

```
    }
  }

  setIsProcessing(true);

  try {
    // Backend ko order bhej rahay hain
    const response = await fetch('http://192.168.0.113:8080/place-order', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        customerName: name || "Yashfa",
        orderItems: cart,
        totalAmount: totalPayment,
        paymentMethod: selectedPaymentMethod
      }),
    });
```

3.4 Summary of API Endpoints:

Method	Endpoint	Port	Respnsibility
GET	/menu	5050	Fetching real-time food item data
POST	/place-order	8080	Submitting customer cart and payment data

3.5 Order Confirmation & Backend State Management:

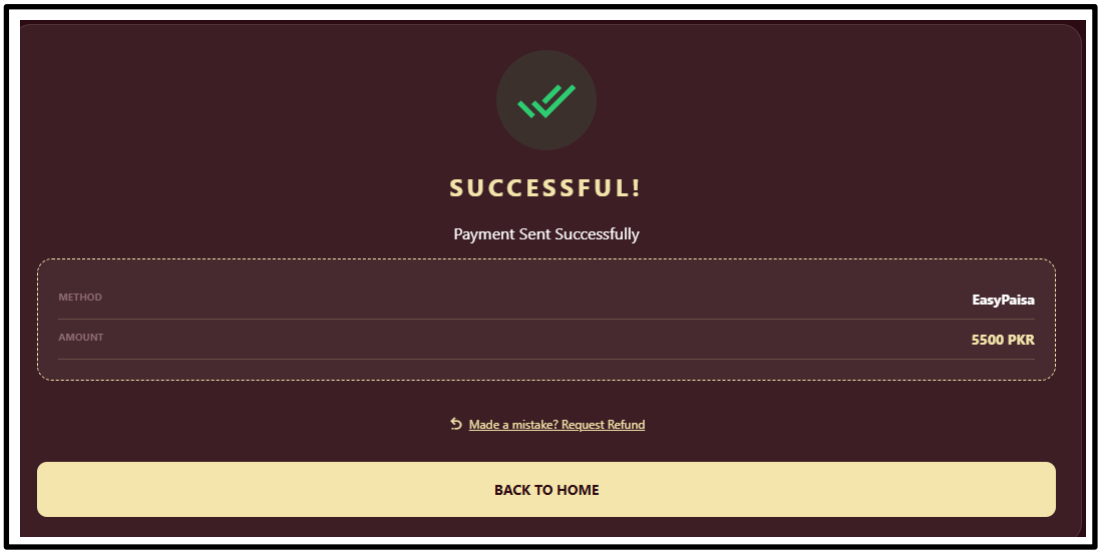
This part of the implementation handles the data flow once the user interacts with the "Confirm Order" button in the app.

Trigger Mechanism: When the user selects items and clicks the checkout button, the frontend validates the cart state.

Payload Structure: The app packages the data into a JSON object. This includes the OrderedItems, TotalTax, and FinalAmount.

Backend Processing: Upon receiving the POST request at Port 8080, the backend logic executes a callback. This callback confirms the data integrity and prints a "Transaction Successful" log in the terminal.

Real-time Feedback: Once the backend confirms receipt (Status 200), the frontend state is updated to clear the cart and navigate the user to a 'Success' screen, completing the full-stack cycle.



```
-------------------------------------
YAHOO! Naya Order Mila Hai:
Customer: Yashfa
Items: [
  {
    id: 106,
    name: "Leo's Special",
    price: 300,
    img: 'https://images.pexels.com/photos/302899/pexels-photo-302899.jpeg'
  },
  {
    id: 105,
    name: 'Green Tea',
    price: 100,
    img: 'https://images.pexels.com/photos/1638280/pexels-photo-1638280.jpeg'
  },
  {
    id: 905,
    name: 'Tea Party',
    price: 600,
    img: 'https://images.pexels.com/photos/1417945/pexels-photo-1417945.jpeg',
    desc: '4 Kashmiri Chais + Biscuits',
    tag: 'EVENING',
    color: '#20B2AA'
  },
  {
    id: 904,
    name: 'Steak Night',
    price: 4500,
    img: 'https://images.pexels.com/photos/3535383/pexels-photo-3535383.jpeg',
    desc: '2 Steaks + 2 Desserts + Mocktails',
    tag: 'PREMIUM',
    color: '#D4AF37'
  }
]
Total Bill: 5500 PKR
-------------------------------------
```

4.Conclusion:

In conclusion, this project successfully demonstrates a full-stack integration between a React Native mobile application and a Node.js backend architecture. By implementing a decoupled service model, the application efficiently fetches dynamic menu data from Port 5050 and securely logs transactions through a dedicated Order Logger on Port 8080.

The successful transmission of data—as evidenced by the terminal logs—proves that the system is capable of handling real-time client-server interactions. This architecture provides a solid foundation for future enhancements, such as database integration (MongoDB/SQL) and secure user authentication.

5.Future Scope (Optional but Recommended):

To further improve this system, the following features can be implemented:

- **Database Integration:** Moving from console logging to persistent storage.
- **Payment Gateway:** Integrating Stripe or PayPal for real-time transactions.
- **Push Notifications:** Alerting the user once the order is prepared.

GitHub Link:

https://github.com/yashfajaved/Food-App-FullStack-Integration/tree/main